

Audion VS Engine Lite

Portable Windows-инструментарий для **технической видеоподготовки и стилизации** на базе VapourSynth + FFmpeg. Lite-сборка = классические VS-плагины, **без ML-стека. 22 пресета в 4 независимых палитрах:**

Палитра	Пресетов	Что делает
Precision Engine	8	Шумодав (включая SOTA-таргетинг теней через BM3D), дебанд, контролируемое восстановление зерна. Технический слой ДО колориста.
Film Looks Engine	5	Эмуляции киноплёнки: 35mm Kodak (250D/500T/50D), 16mm, Super 8, Bleach Bypass, универсальный Cinematic.
Retro Engine	2	Аналоговый характер: VHS / CRT, Camcorder 90s.
Restoration Engine	7	«То, чего нет в Adobe / DaVinci из коробки»: QTGMC, TIVTC, MVTools-MCDeGrain, derainbow, deblock, dehaze плюс NNEDI3/ZNEDI3 2x non-ML upscale.

Движок — **CLI-orchestrator на Python**, оборачивающий пайплайны `vspipe | ffmpeg`.

Лаунчеры — батники с FZF + CMD fallback (английская и русская версии).

Lite **не включает** ML-стек (Real-ESRGAN / RIFE / DPIR) — он живёт в полной сборке `Audion VS Engine`. Lite **не включает** также 18.C-добавки (`imax_70mm`, `anamorphic_scope`, `polaroid`). Lite — это lean baseline (~1.2 GB рабочий объём после установки) для тех, кому ML не нужен.

Первый запуск: доустановить движок

В раздачу не входят VapourSynth и его плагины. Распространять их мы не вправе — это около семидесяти модулей, у каждого своя лицензия. Программа ставит их сама, с сайтов авторов, в

пару нажатий.

Пока это не сделано, **программа ничего не обрабатает**. Она запускается, но движка за ней нет.

Запустите `builder_main.cmd` (или Start → меню сборки) и выберите по порядку:

№	Пункт меню	Что ставит
10	VAPOURSYNTH	сам движок
11	VS PLUGINS	фильтры, которые вызывают пресеты

Порядок важен: плагинам нужен уже установленный движок. Оба шага обязательны, необязательного здесь нет.

FFmpeg и драйвер NVIDIA

Новее не значит лучше. Каждая сборка FFmpeg компилируется под конкретную версию заголовков NVENC, и каждая из них требует своего минимума драйвера. Поставьте самую свежую на драйвер постарше — аппаратное кодирование не ускорится, а перестанет работать.

Сборка FFmpeg	Заголовки NVENC	Минимальный драйвер NVIDIA (Windows)
9.0.1	ffnvcodec n13.1.15.0	610.0
8.0.1	ffnvcodec n13.0.19.0	570.0
7.1.1	ffnvcodec n13.0.19.0	570.0
7.1	ffnvcodec n12.2.72.0	551.76

Обратите внимание на третью строку: 7.1.1 собрана теми же заголовками, что и 8.0.1, и требует те же 570.0 — «откатиться на версию назад» на старом драйвере не даёт ничего. Помогает переход на 7.1 без патча.

Поэтому установщик подбирает сборку по вашему драйверу, а не берёт последнюю. Версии выше прочитаны из README самой сборки, пороги драйверов — из README nv-codec-headers.

Если видеокарты NVIDIA нет, всё это неважно: ставится последняя сборка, а кодирование идёт на процессоре.

Какая сборка идёт в поставке: 8.0.1. Это осознанный выбор, а не забытое обновление.

Большинство машин для монтажа и кодирования сегодня живут на драйверах примерно с 571 по

609; ветка 610 стоит у считанных единиц. И 8.1.x, и 9.x требуют именно её — поставить их значит заявить аппаратное ускорение NVIDIA и не дать его большинству тех, кому оно обещано. В 8.0.1 есть всё, что используют эти программы, и она работает на тех драйверах, которые у людей действительно стоят.

Первоначальная установка (один раз перед Quick start)

Если дерево проекта только что распаковано / клонировано и `system_core/vapoursynth/` / `Tools/ffmpeg/` ещё пусты — запусти `builder_main.cmd` и пройди пункты в таком порядке:

Stage 1 – Оркестратор Python (auto-runs при первом запуске любого *.cmd, но можно прогнать заранее):

[01] BUILD PORTABLE ENV CMD BUILDER (или [03] INSTALL PORTABLE OFFLINE)

Stage 2 – VS engine stack (отсюда берётся ALL GREEN):

[04] POWERSHELL – portable pwsh 7

* ПРОПУСТИТЬ если `pwsh -v` уже >= 7

[10] VAPOURSYNTH – latest VS stable + свой embedded Python

3.12.x

[11] VS PLUGINS – vsrepo + плагины (требует [10])

[12] FFMPEG – portable ffmpeg

Stage 3 – Verify (имеет смысл ТОЛЬКО после Stage 2):

[71] VERIFY / DOCTOR – запускает doctor.py end-to-end
(live BM3D CUDA invocation на NVIDIA)

Stage 4 – Опционально, очистка кеша:

[70] CLEAN INSTALL CACHE – освобождает архивы
install\download\. При повторной
установке само пере-скачается.

Почему [04] раньше времени даёт FAILURE: doctor.py зондирует весь стек (vspipe, плагины, ffmpeg, опциональный CUDA). До Stage 2 этих бинарников ещё нет — ошибки штатные, не баг.

Про портативный PowerShell ([10]): Windows 10/11 несут только Windows PowerShell 5.1, а скрипты `install/*.ps1` используют синтаксис PS 7+ (ternary, null-coalescing). Поэтому **[10] обязателен, кроме случая когда `pwsh -v` уже показывает 7+** на хосте. `.cmd` wrappers автоматически выбирают системный pwsh 7+ поверх портативного, если оба есть.

В Lite **нет MLRT install-шага** (`[70]` — это очистка кэша, не ML install). Для ML-пресетов используй полную сборку `Audion VS Engine`.

Быстрый старт

1. Распакуй проект на любой диск (он portable, никакой установки).
2. Запусти `launcher_project.cmd` — диспетчер: `[P]` Precision / `[F]` Film Looks / `[R]` Retro / `[N]` Restoration / `[A]` Apply profile к файлу или папке.
3. Положи источник в `input\`, результаты пойдут в `output\`.

Каждый launcher палитры проводит через input → params → encoder → run.

Политика аудио

GUI показывает настройки видеопотока. Аудио обрабатывается автоматически:

- по умолчанию исходная аудиодорожка копируется без перекодирования (`-c:a copy`);
- если входной видеопоток ProRes перекодировается в lossy x264/x265, аудио уходит в AAC 384 kbps для совместимости;
- `--no-audio` остаётся технической CLI-опцией для smoke/benchmark, не основным GUI-сценарием.

Четыре палитры — конкретное содержимое

Precision Engine (`cli\launcher_precision.cmd`)

Pipeline-меню читается сверху вниз как реальный signal flow:

```

=== Stage 1 -- Denoise (clean first) ===
[01] Mild denoise          DFTTest спектральный, light/medium/strong
[02] Shadow denoise SOTA *  BM3D (CUDA->CPU auto) + smooth luma-mask «только тени»
[03] Chroma cleanup        DFTTest по chroma plane'ам

=== Stage 2 -- Deband (smooth gradients) ===
[04] Deband safe           neo_f3kdb light, без зерна
[05] Deband + fine grain   neo_f3kdb + AddGrain микро-восстановление

=== Stage 3 -- Compositions (full chains) ===
[06] Filmic rebuild       *   denoise -> deband -> luma-zoned grain (флагман)
[07] Archive clean        нейтральный мастер, без зерна
[08] Pre-grade prep       минимальное воздействие, handoff в DaVinci Resolve

```

Cross-vendor: `audion_lib.bm3d_auto()` зондирует NVIDIA через `nvidia-smi -L` и сам выбирает BM3DCUDA / BM3DCPU. Intel / AMF работают из коробки.

Film Looks Engine (`cli\launcher_film_looks.cmd`)

5 луков по нарастанию характера (subtle → extreme):

```

[01] Cinematic            универсальный subtle filmic, безопасный default
[02] Film 35mm            варианты Kodak 250D / 500T / 50D
[03] Film 16mm            органичный, более зернистый, поднятые тени
[04] Super 8              тяжёлое зерно, выцветшие 70-е
[05] Bleach bypass        высокий контраст, десатурированный («Se7en»)

```

Все Film Looks делят `system_core/presets/audion_lib.py`: MTF softening, halation bloom, luma-zoned grain, gamma curve, black-lift, desaturation.

Retro Engine (`cli\launcher_retro.cmd`)

```

[01] VHS / CRT            chroma bleed, мягкая оптика, аналоговый шум
[02] Camcorder 90s        мягче VHS, лёгкая переэкспозиция

```

Restoration Engine (`cli\launcher_restoration.cmd`)

Набор «Adobe / DaVinci так не умеют из коробки». Требуется дополнительные плагины (havsfunc, mvsfunc, mvtools, tivtc, znedi3, nnedi3_resample + Python-зависимость `vsutil` для havsfunc) — ставятся автоматически через `Install-VS-Plugins.cmd`.

```

=== Field rebuild ===
[01] QTGMC deinterlace      NNEDI3 + MVTools; reference deinterlace для DV/HDV/VHS
[02] TIVTC inverse-telecine  NTSC 29.97 telecined -> 23.976 progressive (3:2
pulldown removal)

=== Motion-compensated denoise ===
[03] MVTools MCDeGrain      темпоральный шумодав с сохранением деталей (Topaz-
style)
[04] Derainbow / decross     NTSC composite chroma cleanup (rainbow, dot crawl)

=== Compression rescue ===
[05] Deblock H.264 artefacts спасение пережатых YouTube / WhatsApp / SD broadcast
[06] Dehaze / local contrast clarity без halos, без сдвига цвета, 16-bit math

=== Clean upscale ===
[07] NNEDI3 2x upscale       детерминированное non-ML увеличение 2x для Lite

```

Системные требования

- Windows 10/11 x64
- ~1.2 GB свободного места (рабочий объём проекта после распаковки — Lite, без ML)
- Опционально: NVIDIA GPU + driver R525+ для ускорения `bm3dcuda` (10–50× быстрее CPU на BM3D-heavy пресетах). Без NVIDIA `audion_lib.bm3d_auto()` сам уходит на BM3DCPU.
- Системный Python не нужен — embedded Python живёт в `runtime/`.
- Системный FFmpeg не нужен — portable FFmpeg живёт в `Tools/ffmpeg/`.

Если что-то не работает — `launcher_project.cmd` → `[D] Doctor` для диагностики.

Архитектура (коротко)

Проект использует **два независимых embedded Python**:

```

runtime/python.exe          # Оркестратор (latest Python 3.12.x)
system_core/vapoursynth/python.exe  # VS-host (latest Python 3.12.x) --
запускает vspipe + плагины

```

Оркестратор никогда не импортирует VapourSynth. Он зовёт `system_core/vapoursynth/Scripts/vspipe.exe` через subprocess и пайпит у4м в `Tools/ffmpeg/bin/ffmpeg.exe`.

Важно для VS R74+: реальная директория автозагрузки плагинов берётся через `vapoursynth.get_plugin_dir()` и в wheel-layout находится внутри `Lib\site-packages\vapoursynth\plugins\`. Старая `system_core\vapoursynth\vs-plugins\` — legacy; она может быть пустой и не должна использоваться для install/status.

```
launcher_*.cmd → runtime/python.exe system_core/main.py run \
    --palette X --preset Y --input ... --output ...
↓
↓ subprocess: vspipe -c y4m preset.vpy - | ffmpeg -i - ... output
↓
↓ все .vpy пресеты читают параметры из AUDION_VS_* env vars
↓
↓ JSON-отчёт пишется в logs/{ts}__{palette}__{preset}__{stem}.json
```

Набор плагинов (auto-install через `Install-VS-Plugins.cmd`):

- v1.0 set: `lsmas`, `ffms2`, `fmtconv`, `neo_f3kdb`, `addgrain` (`grain` namespace), `knlmeanscl` (`knlm`), `bm3dcpu`, `dfttest`
- `bm3dcuda` (NVIDIA-ускорение; **по умолчанию ставится**, opt-out через `/NO-CUDA`)
- Restoration set: `havsfunc`, `mvsvfunc`, `mvtools`, `tivtc`, `znedi3`, `nnedi3_resample` (+ `vsutil` из PyPI для havsvfunc)

CLI-справочник

```
runtime\python.exe system_core\main.py <команда> [args]
```

Команда	Что делает
<code>info</code>	Печатает резолвнутые пути, версии Python, количество namespace плагинов
<code>doctor</code>	Запускает <code>system_core/doctor.py</code> — полный health-check (Pythons, vspipe, плагины, ffmpeg, опциональный live-CUDA smoke)

Команда	Что делает
<code>list-presets</code>	Все 21 зарегистрированных пресета по 4 палитрам, с дефолтами параметров и группировкой
<code>list-encoders</code>	Все 34 профиля энкодера (software CRF / QSV / NVENC / AMF / ProRes / DNxHR)
<code>list-profiles</code>	Встроенные (5) + пользовательские профили из <code>config\profiles*.json</code>
<code>materialize-profiles [--force]</code>	Записать 5 встроенных профилей в <code>config\profiles\</code> как редактируемый JSON
<code>probe --input X</code>	ffprobe-сводка (codec, разрешение, fps, длительность, аудио, color metadata)
<code>run --palette P --preset Q --input I --output O [params...]</code>	Полный pipeline обработки
<code>apply-profile --name N --input I --output O [--no-audio]</code>	Запуск сохранённого профиля на одном файле
<code>apply-profile-batch --name N --input-dir D --output-dir E [--recursive] [--no-mirror] [--overwrite]</code>	Профиль по папке; <code>--recursive</code> обходит подпапки, зеркалит структуру, пропускает уже обработанное

Общие флаги `run`: `--strength {light,medium,strong}`, `--sigma <float>`, `--use-cuda 0|1`, `--grain-back <float>`, `--shadow-threshold <0..1>`, `--transition <0..1>`, `--deband-range <int>`, `--grain-shadow / --grain-mid / --grain-high <float>`, `--high-threshold <0..1>`, `--stock {250D,500T,50D}` (film_35mm), `--intensity <float>`, `--field-order {tff,bff}`, `--qtmcmc-preset {Faster..Placebo}`, `--output-fps {single,double}`, `--radius <int>`, `--thsad <int>`, `--blksize <int>`, `--quant1 / --quant2 <int>`, `--encoder <profile>`, `--no-audio`. Полный список — `... main.py run --help`.

Профили энкодеров (34 шт., ladder 14/17/21): software `h264_crf{14,17,21}` / `h265_crf{14,17,21}` (default `h264_crf14`); Intel QuickSync `h264_qsv_q{14,17,21}` / `h265_qsv_q{14,17,21}`; NVIDIA NVENC `h264_nvenc_q{14,17,21}` / `h265_nvenc_q{14,17,21}`; AMF `h264_amf_q{14,17,21}` / `h265_amf_q{14,17,21}`; ProRes `prores_lt` / `prores_lt_mxf` / `prores_422` / `prores_422_mxf` / `prores_422hq` / `prores_422hq_mxf`; DNxHR `dnxhr_lb/sq/hq/hqx`.

Профили (сохранённые комбинации)

Встроенные профили в Lite v1.0:

Имя	Что
<code>shadow_clean_quick</code>	Быстрая чистка теневого шума на тёмной цифровой съёмке
<code>filmic_warm_35mm</code>	Kodak 250D 35mm filmic look на intensity 1.0
<code>archival_master</code>	Нейтральный архивный мастер, ProRes LT (или HQ для keying)
<code>resolve_handoff</code>	Минимальная подготовка, ProRes LT (MXF wrapper доступен) для DaVinci / Avid
<code>vhs_dreamy</code>	VHS/CRT retro на intensity 1.2

```
runtime\python.exe system_core\main.py list-profiles
runtime\python.exe system_core\main.py apply-profile --name filmic_warm_35mm ^
--input input\source.mov --output output\filmic.mp4
```

Встроенные **auto-materialize** в `config\profiles\` как редактируемый JSON при первом `list-profiles` / `apply-profile`. Свои профили: скопируй любой `config\profiles\*.json` под новым именем, отредактируй `params` / `encoder` / `description` — он сразу появится в `list-profiles`.

Batch по папке:

```
runtime\python.exe system_core\main.py apply-profile-batch ^
--name filmic_warm_35mm --input-dir input\shoot_2026_05 ^
--output-dir output\shoot_2026_05_filmic --recursive --no-audio
```

Файлы именуются `<stem>__<profile>.mp4`. `--recursive` зеркалит дерево; уже обработанные пропускаются (idempotent restart-safe).

Diagnostic & maintenance скрипты

`builder_main.cmd` — точка входа в меню. Карта пунктов:

#	Пункт	Скрипт
[01..09]	Build / licenses / release	template-owned
[04]	Установить portable PowerShell	Install-Portable-PowerShell.cmd
[10..12]	Установить VS / VS plugins / FFmpeg	Install-Portable-*.cmd, Install-VS-Plugins.cmd
[70] ★	Clean install cache — чистит transient install downloads, staging dirs и bytecode caches, сохраняя portable payloads	Clean-Install-Cache.cmd
[90+]	Project launcher / open install/runtime/wheels/licenses/release	—

В Lite **нет** [13] VS-MLRT LEAN / [14] VS-MLRT FULL — это фишки полной сборки. У Lite [70] CLEAN INSTALL CACHE — это очистка кэша.

Standalone скрипты в `install/`:

- **Repair-PipShims.cmd** — чинит shebang в `Scripts\*.exe` после переезда проекта. Авто-вызывается из `system_core/engine/selfheal.py` на старте `main.py` / `doctor.py`; вручную нужен только если ходишь напрямую в `vspipe.exe` мимо оркестратора. `/WHATIF` (или `/N`) — dry-run.
- **Bench-CUDA.cmd** — pure-pipeline (`vspipe → ffmpeg -f null`) тайминг CPU vs CUDA на `shadow_denoise_sota` и `filmic_rebuild`. Drag-drop файла. Эталон на RTX 5070 / Ryzen 9 5900X / DCI 4K ProRes 25 c: `shadow_denoise_sota` -35%, `filmic_rebuild` -24%.
- **Bench-AllPresets.cmd** ★ — smoke-walker по всем 22 пресетам. Drag-drop файла; usage: `Bench-AllPresets.cmd <video> [frames=30] [cuda_mode=off|on|sweep]`. Пишет **Total** / PASS / FAIL + JSON-отчёт `logs\bench_all_presets_<TS>.json`. Свежий smoke: **22/22 PASS** (Frames=1, Cuda=off, 2026-05-16).
- **Clean-Install-Cache.cmd** — см. `builder_main → [70]`.
- **Ensure-7zip.ps1** — dot-source helper: portable `7zr.exe` / `7za.exe` ставятся уже на `builder_main → [01]/[02]` вместе с Python в `system_core\7zip\`. Используется `Install-Portable-VapourSynth.ps1` и `Install-Portable-FFmpeg.ps1` для быстрого распаковывания больших архивов (`Expand-7zArchive` ~3× быстрее `Expand-Archive` на multi-GB ZIP'ах и не давится на >2 GB).

- `runtime\python.exe system_core\doctor.py` — полный health-check, включает live `core.bm3dcuda.BM3D(...)` invocation как авторитетный CUDA-сигнал.

Портативность

Проект **полностью portable**. Скопируй папку на любой Windows-диск или машину — работает. Никакого инсталлера, системного Python, изменений PATH/реестра — все компоненты (оба embedded Python, VS-host, ffmpeg, fzf, portable PowerShell 7, 7zr.exe) живут внутри `system_core/`. После переезда папки следующий запуск `runtime\python.exe system_core\doctor.py` (или любого `main.py`) сам чинит pip-launcher shebang'и. Единственная out-of-tree зависимость — NVIDIA video driver, если хочешь CUDA.

CUDA setup (только NVIDIA)

`bm3dcuda` — опциональный ускоренный BM3D плагин. Минимум:

- **NVIDIA Studio Driver R525+** (Game Ready тоже работает; Studio предпочтительнее) — драйвер сам несёт `cuda64_12.dll` и `cufft64_*.dll`, всё что нужно `bm3dcuda` в runtime.
- **CUDA Toolkit Network installer "Runtime libraries" only (~300 MB)** — нужен в редких случаях, когда driver-bundled runtime не совпадает с build плагина (например, на Blackwell sm_120 пригодился Toolkit 13.2.3 для чистой PTX JIT-компиляции).

Полный CUDA SDK / cuDNN / TensorRT — **не нужны**.

`Install-VS-Plugins.cmd` ставит `bm3dcuda` по умолчанию. `/NO-CUDA` — пропустить на non-NVIDIA. Затем `runtime\python.exe system_core\doctor.py` для проверки live-вызовом.

Где CUDA раскрывается сильнее

- **Разрешение** важно: 4K даёт в 1.5–2× больший gain чем 1080p (BM3D = O(пикселей), GPU launch overhead — константа).
- **Длина клипа** важна: до ~5 секунд JIT/setup не амортизируется.
- **Вес пресета** важен: `shadow_denoise_sota` и `filmic_rebuild` — почти весь wall-time это BM3D, gain заметный. `mild_denoise`, `chroma_cleanup`, `deband_*` живут на DFTTest / neo_f3kdb (CPU-only) — CUDA не помогает.

Эталон на RTX 5070 + Ryzen 9 5900X (pure denoise, без encode):

- 1080p × 32 с H.264: shadow_denoise_sota –22%, filmic_rebuild –15%
 - DCI 4K × 25 с ProRes: shadow_denoise_sota **–35%**, filmic_rebuild **–24%**
-

Структура проекта

Audion VS Engine Lite/	
├ launcher_project.cmd launcher_project_ru.cmd	Top-level диспетчер
├ cli/	CLI-лаунчеры палитр
└ launcher_precision.cmd launcher_precision_ru.cmd	Stage 1/2/3 pipeline (8
пресетов)	
└ launcher_film_looks.cmd launcher_film_looks_ru.cmd	5 film looks
└ launcher_retro.cmd launcher_retro_ru.cmd	2 retro looks
└ launcher_restoration.cmd launcher_restoration_ru.cmd	6 restoration пресетов
├ builder_main.cmd launcher_gui.cmd launcher_tools.cmd	Сервисные/GUI-лаунчеры
├ runtime/	Embedded Python
оркестратор (latest 3.12.x)	
├ system_core/	
└ main.py doctor.py	CLI + диагностика
└ engine/	runner / env / probe /
logging / presets / profile / selfheal	
└ presets/{precision,film_looks,retro,restoration}/	21 .vpy preset
└ vapoursynth/	VS-host (свой latest
Python 3.12.x + плагины)	
└ presets/audion_lib.py	shared preset helpers
(incl. bm3d_auto)	
└ ffmpeg/	Portable FFmpeg
(BtbN/Gyan GPL)	
└ powershell/	Portable PowerShell 7
└ 7zip/7zr.exe	Portable 7-Zip CLI
└ fzf.exe	FZF binary
├ install/	Установщики + diagnostic
скрипты (.cmd + .ps1):	
	Install-Portable-
{PowerShell,VapourSynth,FFmpeg}, Install-VS-Plugins,	
	Clean-Install-Cache (~700
MB reclaim),	
	Ensure-7zip (dot-source
helper для 7zr/7za),	
	Repair-PipShims (auto-
invoked from selfheal),	
	Bench-CUDA (pure-denoise
CPU vs CUDA bench),	
	Bench-AllPresets (smoke
по всем 22 пресетам),	
	make_release_archive

```
(release zip с dev-artefact exclusions)
```

```
├─ GitHub/
```

Publication-ready docs

```
(эта папка)
```

```
├─ config/
```

Defaults + user profiles

```
(`profiles/*.json`)
```

```
├─ input/  output/  logs/  release/
```

User folders

```
└─ CLAUDE.md  MEMORY.md
```

Контекст для AI-агентов

Документация

- **CLAUDE.md** — рабочий контракт для AI-агентов, продолжающих разработку (корень)
- **MEMORY.md** — полное состояние проекта, архитектурные решения, gotchas, фиксы (корень)
- **GitHub/README_EN.md** / **README_RU.md** — landing page (этот файл)
- **GitHub/VapourWiki_RU.md** — детальный справочник по всем 22 пресетам и энкодерам (decision tree, ключевые параметры)
- **GitHub/SECURITY.md** — security policy
- **LICENSE** / **GitHub/LICENSE (GPL-3.0-or-later).md** — текст GPLv3 для проектной лицензии (**GPL-3.0-or-later**)
- **GitHub/** прочие файлы — publication-meta (release notes, project page description, one-liner)

Лицензия

Авторский код Audion, скрипты, лаунчеры, пресеты и документация лицензируются как **GPL-3.0-or-later** (см. **LICENSE**). Сторонние инструменты и библиотеки (VapourSynth, FFmpeg, плагины, Python, wheels, PowerShell, 7-Zip, fzf) остаются под собственными лицензиями; собирай их через **builder_main.cmd** → **[06] Collect release licenses** в **licenses/** и **licenses/THIRD_PARTY_NOTICES.md**.

Статус: Lite v1.0 + Phase 18.B Restoration + NNEDI3 2x ☒ — production-ready на Windows 10/11 x64. Свежий локальный all-presets smoke: **22/22 PASS** (**Frames=1**, **Cuda=off**, 2026-05-16). Intel encoder smoke остаётся **22/22 доступных профиля PASS** от 2026-05-10; NVENC/AMF требуют соответствующее железо. CUDA sweep для этой 22-preset точки — handoff на NVIDIA-машину: **Bench-AllPresets.ps1 -Frames 1 -Cuda sweep**.

Канонические названия Workbench

Workbench использует единый публичный словарь Audion Image Tools во всех проектах. Кнопки всегда расположены и называются одинаково: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

Сбросить возвращает проектные **input/output** и не удаляет файлы; **Удалить** очищает текущие **Источник** и **Назначение** только после подтверждения. В английском интерфейсе точные названия: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. Варианты **Цель**, **Очистить**, **Destination** и **Clear** для этих элементов Workbench не используются.